Um Modelo para Gerenciamento de Perfis de Entidades Através de Inferência em Trilhas

André Wagner

1 Introdução

Atualmente, as aplicações têm se concentrado na coleta de informações a respeito dos usuários, com o objetivo de permitir que elas possam conhecer suas preferências, possibilitando a oferta de serviços personalizados. Assim, estas aplicações organizam seu conhecimento a respeito do usuário em *perfis de usuário*, onde cada uma salva as informações em um formato baseado em um *modelo de usuário* específico.[?]

Sistemas sensíveis a contexto são aplicações nas quais o contexto atual de um usuário é levado em consideração na disponibilização de serviços. Dey[?] define contexto como qualquer informação que pode ser utilizada para caracterizar a situação de uma entidade, onde uma entidade é uma pessoa, local ou objeto considerado relevante à interação entre o usuário e a aplicação. Desta forma, o *perfil* torna-se parte essencial na descoberta de informações pertinentes aos usuários que possam ser utilizadas na construção de um contexto.

A capacidade de uma aplicação compreender o contexto de uma entidade pode ser amplificada através do uso de *trilhas*[?]. Uma trilha é o histórico de contextos visitados por uma entidade, e das ações realizadas pela entidade no contexto.

A coleta de informações de um usuário pode ser *explícita* ou *implícita*[?]. Na coleta *explícita*, é requisitado ao usuário que preencha informações através de um formulário pré-definido, enquanto na coleta *implícita*, a informação é inferida a partir do estudo do comportamento dos usuários.

Informações a respeito de um usuário colhidas por uma aplicação podem ser úteis para outras aplicações que tenham o mesmo usuário [?]. No entanto, os diferentes *modelos de usuário* utilizados pelas aplicações tornam difícil a utilização de informações de um *perfil de usuário* de uma aplicação por outra. Além disso, algumas das informações do usuário podem ser válidas apenas dentro de um *contexto* específico, o que dificulta ainda mais que diferentes aplicações compartilharem o mesmo perfil.

Como forma de ultrapassar estas barreiras, o conceito de *personalização inter-sistemas* foi introduzido [?] como forma de compartilhar informações de usuários entre diferentes aplicações. Como o maior problema deste tipo de modelo é o alinhamento entre as diferentes abordagens de personalização, o estudo de *sistemas de modelos de usuário* é o principal desafio da pesquisa atual na área de personalização.[?]

O IEEE Standard Computer Dictionary [?] define interoperabilidade como "a capacidade de dois ou mais sistemas ou componentes trocarem informações e utilizar a informação que foi trocada." No âmbito da personalização, interoperabilidade é a capacidade de acessar e interpretar informação derivada de múltiplas fontes heterogênias e integrar esta informação em um modelo de usuário de granularidade própria[?]. Os dois principais modelos de interoperabilidade são: (i) *interoperabilidade sintática*, onde as diferenças entre os modelos de usuário são resolvidos em nível de aplicação, e (ii) *interoperabilidade semântica*, onde as diferenças entre os modelos são resolvidos em nível de conhecimento.

As duas principais abordagens utilizadas em sistemas de modelos de usuários interoperáveis são [?]:

Baseados em padrões : são baseados numa visão *top-down*, definindo um padrão que todas as aplicações envolvidas devem observar;

Baseados em mediação : se baseiam numa visão *bottom-up*, onde ocorre algum tipo de *reconciliação* entre os diferentes modelos de re-

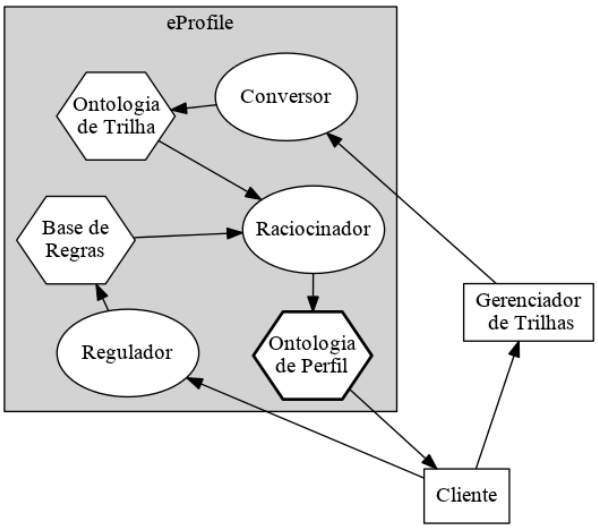

Figura 1: Modelo do eProfile

presentação.

2 eProfile

O eProfile é um gerenciador de perfis voltado para ambientes distribuídos, que infere dados de perfil através da inferência das trilhas de uma entidade. A trilha é disponíbilizada por um gerenciador de trilhas que se conecta ao eProfile, onde é feita uma mediação entre o formato de dados da trilha e do eProfile.

A ontologia de perfil do eProfile pode ser extendida pelos clientes, de modo que os clientes possam armazenar dados de perfil que interessem apenas a si mesmos. Assim, os clientes podem disponibilizar suas próprias regras para a inferência das informações relacionadas à sua própria estrutura de dados. Os dados inferidos ficam então disponíveis para serem consultados pelas entidades.

2.1 Modelo

A Figura 1 apresenta o modelo do eProfile.

A figura tradicional do usuário na personalização é extendida aqui pelo conceito de *entidade*. Uma entidade é semelhante a um usuário, porém em um conceito mais amplo, não representando apenas pessoas mas também objetos que possam criar uma trilha e ter um perfil, como um veículo ou um

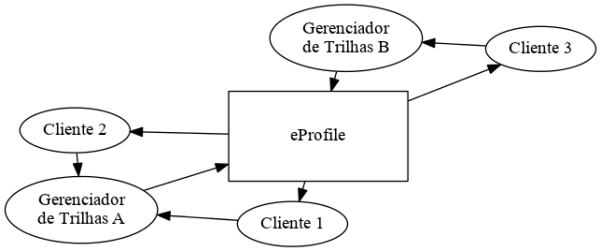

Figura 2: eProfile conectado a mais de um gerenciador de trilhas

dispositivo móvel. Uma entidade pode também ser composta por um grupo de entidades como, por exemplo, uma frota de caminhões onde cada caminhão tem seu perfil único, e onde a frota tem um perfil também.

Os *clientes* são os aplicativos que são utilizados pelas entidades para se conectar aos gerencidadores de trilhas e perfil. Neste modelo, o cliente se conecta a um gerenciador de trilhas genérico, que é responsável pelo armazenamento da trilha da entidade. O perfil da entidade será inferido a partir das trilhas registradas no gerenciador.

As trilhas das entidades são buscadas do gerenciador de trilhas pelo eProfile. Como o eProfile pode se conectar a diferentes gerenciadores de trilha, onde em cada um deles a trilha tem um formato específico (ver Figura 2), é necessário que seja realizada uma conversão entre o formato original e o formato utilizado pelo eProfile, que será apresentado mais adiante. Este trabalho é realizado por um agente entitulado *conversor*, que é capaz de realizar a conversão para os formados de trilha suportados pelo eProfile.

Desta forma, o eProfile não tem a necessidade de conhecer o cliente – mas precisa conhecer o formato utilizado pelo gerenciador de trilhas.

A trilha do usuário é então armazenada dentro do eProfile em uma *ontologia de trilha básica*. Esta ontologia possui as classes básicas para a construção de uma trilha, sendo que a mesma pode ser extendida de acordo com a especificação do gerenciador de trilhas que está sendo utilizado. A forma como ocorre esta extensão é detalhada na Seção 2.2.

Como cada cliente (aplicação) tem necessidades próprias de inferência – isto é, cada cliente tem interesse em um conjunto de informações de perfil – os clientes são os responsáveis pelo envio das regras

de inferência ao eProfile. Assim, o cliente envia as regras ao gerenciador de trilhas, que por sua vez, as envia ao eProfile. Mas o cliente envia as regras diretamente ao eProfile.

O agente responsável pela recepção das regras é o *regulador*. O regulador tem a função de receber as regras, validá-las e inserí-las na *base de regras*. Juntamente com as regras, é especificado a criticidade da informação. O sistema utilizará esta informação para determinar com que freqüência a inferência desta informação será executada.

Uma vez de posse das trilhas e das regras, o agente *raciocinador* realiza a inferência da trilha, gerando assim a *ontologia de perfil*. Esta ontologia, da mesma forma que a ontologia de trilha, contém classes básicas a partir das quais os aplicativos vão especializar, utilizando-as para armazenar informações que sejam úteis a eles. O cliente pode então conectar-se ao eProfile e buscar o perfil de sua entidade.

2.2 Ontologias

O eProfile utiliza internamente duas ontologias: uma ontologia de trilhas e uma ontologia de perfis. Para o armazenamento e processamento das ontologias, foram estudadas algumas linguagens de representação de conhecimento.

A linguagem **CycL** é a linguagem utilizada pelo o projeto de *Cyc*[?]. O Cyc é um projeto de inteligência artificial que tem o objetivo de montar uma ontologia e base de conhecimento a partir do senso comum, com o propósito de permitir que aplicações de inteligência artificial realizem raciocínio semelhante ao humano. A linguagem CycL é uma linguagem declarativa baseada em lógica de primeira ordem clássica, com extensões para operadores modais e lógica de ordem superior. A base de conhecimento é dividida em micro-teorias (Mt), que são coleções de conceitos e fatos tipicamente pertencentes a um domínio particular de conhecimentos e, ao contrário da base de conhecimento como um todo, cada micro-teoria deve estar livre de contradições.

RDF (*Resource Description Framework*) é uma linguagem utilizada para representação de metadados a respeito de recursos na Web[?]. No entanto, o conceito de "recurso da Web" pode ser generalizado, de forma que o RDF pode ser utilizado para representar informações a respeito de coisas que podem ser identificadas na Web, mesmo que elas não possam ser carregadas da Web. Recursos podem ser recursos eletrônicos, como arquivos, ou conceitos, como pessoas. O RDF é complementado pelo **RDF/S** (*RDF/Schema*), uma extensão semântica do RDF. O RDF/S é uma linguagem de descrição de vocabulário que provê mecanismos para descrever grupos de recursos relacionados e os relacionamentos entre estes recursos[?].

O **DAML+OIL** é uma linguagem de representação de conhecimento baseada nas linguagens DAML e OIL. O DAML (*DARPA Agent Markup Language*) foi desenvolvido pelo Centro de Pesquisas Avançadas da Defesa Americana e é uma linguagem de marcação de agentes baseada no RDF. Já o OIL (*Ontology Inference Layer*) é uma linguagem com os mesmos objetivos, criada pelo por um grupo de universidades europeias. Ambos os esforços foram unidos na criação do DAML+OIL, que por sua vez deu origem ao OWL[?].

O **OWL** é a linguagem de ontologia mais expressiva definida ou em definição para a Web Semântica[?]. O OWL utiliza URI[1] para os nomes, e a estrutura de descrição disponibilizada pelo RDF. O OWL está uma camada acima do RDF e do RDF/S, e adiciona mais vocabulário para descrever propriedades e classes: entre outros, relações entre classes, cardinalidade, igualdade, tipagem mais avançada de propriedades, características de propriedades e classes enumeradas[?]. O OWL está disponível em três sublinguages: *OWL Lite*, *OWL DL* e *OWL Full*, cada uma aumentando o nível de complexidade e expressividade. Um grande número de ferramentas com capacidade para edição e raciocínio semântico com suporte para OWL está disponível no mercado.

Devido à expressividade e ao suporte de ferramentas externas, o OWL foi escolhido como linguagem de ontologias utilizado pelo eProfile para modelagem da ontologia do perfil.

As ontologias utilizadas pelo eProfile – tanto a ontologia de trilhas quanto a ontologia de perfil – são ontologias básicas, extendidas através de subclasses de acordo com o formato da trilha enviada pelo gerenciador (no caso da ontologia de trilha) ou das necessidades do cliente (no caso da ontologia do perfil). Isto é representado na Figura 3, onde o eProfile oferece uma classe genérica "Evento", que é especializada pela classe "EventoDeCompra", uti-

[1] *Uniform Resource Identifier*

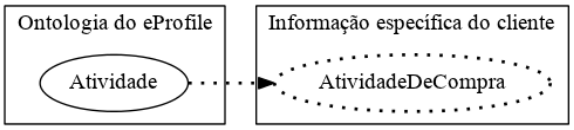

Figura 3: Exemplo da extensão de uma classe do eProfile

lizada por um dos gerenciadores de trilha.

2.2.1 Ontologia de Trilha

A ontologia de trilha está representada na Figura 4, onde todos os relacionamentos, representados pelas setas, são do tipo "has-some" (tem-algum(s)).

Abowd e Dey definem que aplicações sensíveis a contexto estão interessadas em saber *quem*, *onde*, *quando* e *o quê* (isto é, o que o usuário está fazendo), de forma a poderem descobrir por que a situação está ocorrendo[?]. Assim, uma trilha é representada por um conjunto de eventos caracterizados pelas seguintes classes:

Indentity: É a identidade da entidade que está realizando a ação. Esta entidade pode estar realizando a ação com um ou mais recursos (*Resource*) (como um aplicativo ou dispositivo) e pode estar vinculada a um perfil (*Profile*) dentro do eProfile. A classe **Participants** contém as identidades das entidades que estão participando deste evento;

Location: Onde está ocorrendo a ação. Uma localização pode ser uma localização física específica (como uma sala) ou não (uma estrada, por exemplo). A localização pode estar vinculada a uma região (*Region* (que, adicionalmente, pode estar vinculada a outras subregiões) e geralmente possui uma localização global (GPS) vinculada a ela;

Time: Reponde à questão de *quando* ocorreu o evento;

Activity: Refere-se à ação que o usuário está executou (ou tentou executar). É a classe mais abrangente da ontologia, e a que possivelmente será a mais especializada pelos gerenciadores de trilha.

É importante notar que a própria classe base *Thing*[2] pode ser extendida, como forma de registrar outras informações necessárias. Por exemplo, a trilha registra o interesse de um usuário em um determinado produto, a classe *Thing* pode ser extendida à classe *Product* para o registro das informações deste produto.

2.2.2 Ontologia de Perfil

A ontologia de perfil é a ontologia utilizada para armazenar as informações de perfil inferidas a partir da trilha. Esta ontologia está representada na Figura 5.

A ontologia possui duas classes básicas: *Entity* (entidade) e *Information* (informação). Assim, os perfis de várias entidades podem ser armazenados nesta ontologia, onde cada perfil possui um conjunto de informações. Este conjunto de informações será extendido pelas aplicações através de herança (subclasses).

Cada entidade é identificada por um nome único, mesmo que o perfil seja alimentado por vários clientes. Desta forma, é possível que um cliente tenha acesso a informações de perfil gravadas por outro cliente, desde que ele tenha conhecimento das classes utilizadas pelo outro.

As informações podem ser de três tipos:

PastInformation: refere-se a eventos que ocorreram no passado. As informações armazenadas nesta classe são, de um modo geral, cumulativas. Exemplos seriam as escolas onde uma pessoa estudou, ou os destinos onde um caminhão já entregou alguma carga;

PresentInformation: são informações atuais da entidade, subdivididas em subclasses descritas mais adiante;

FutureInformation: são informações previstas a respeito do futuro de uma entidade, como o conjunto de disciplinas que uma pessoa tem a cursar, ou quantos quilômetros um caminhão pode percorrer antes de reabastecer.

As informações atuais podem ser dos seguintes tipos:

[2]Em OWL, a classe *Thing* é a classe base a partir da qual todas as outras classes são extendidas.

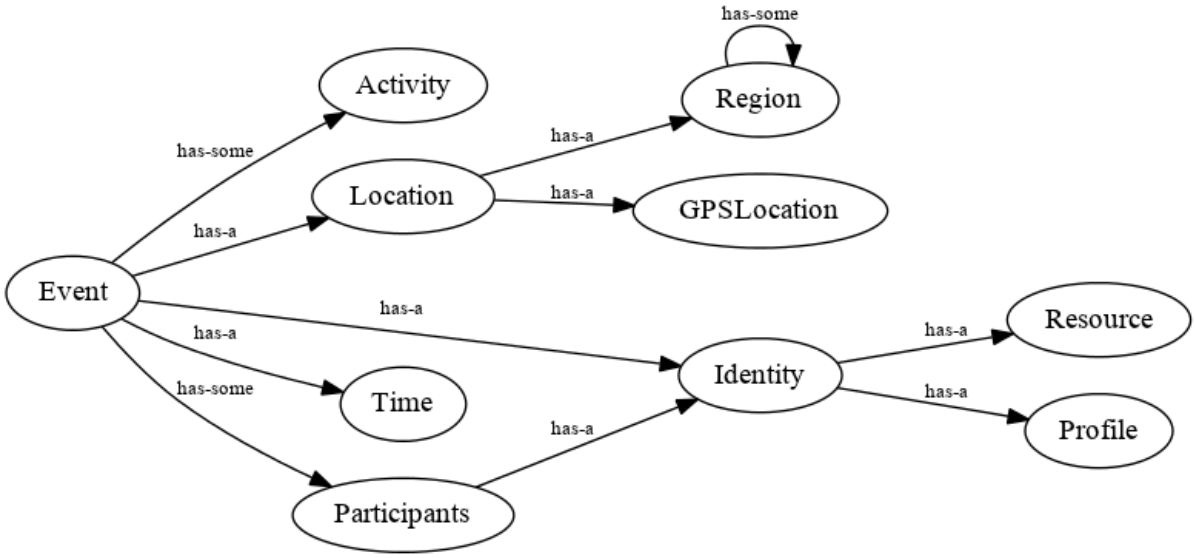

Figura 4: Ontologia de Trilha do eProfile

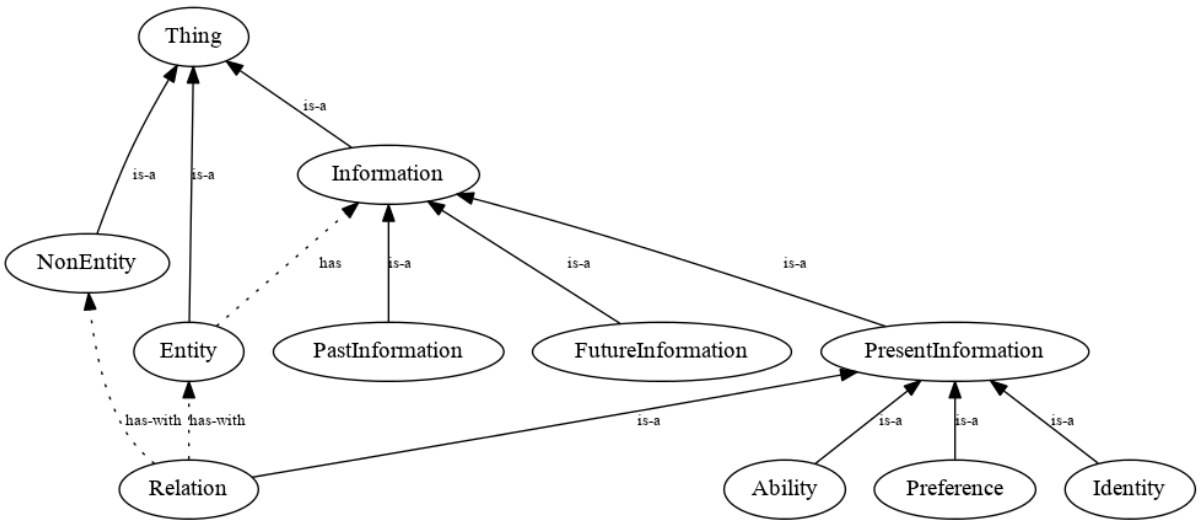

Figura 5: Ontologia de Perfil do eProfile